

UX Harness Framework v3.1

Arquitectura de Ingeniería de Contexto y Calidad Automatizada (Quality Gate)

ACTUALIZADO 2026 — Incluye mejoras de versionado, responsividad, validación semántica, navegación avanzada, gestión de activos críticos (mapas), seguridad y SEO.

1. Estructura de Directorios Actualizada

Se ha integrado la capa de **Calidad** y **Versionado** en el núcleo del sistema para asegurar que cada entregable cumpla con los estándares definidos y sea trazable en el tiempo.

```
/ux-harness
agents.md
--core/
  design-system.md
  ux-principles.md
  responsive-spec.md
  performance.md
  seo-rules.md (NUEVO)
--quality/
  linter-rules.md
  accessibility.md
  heuristics.json
--projects/
  -- [cliente-x]/
    -context.md
    -context.schema.json
    -changelog.md
    -.env.example (NUEVO: Seguridad API)
    -feedback.md (NUEVO: Feedback Humano)
  --assets/
  --src/
  --dist/
    -test-report.json
    -audit-summary.md (NUEVO: Reporte Ejecutivo)
```

2. The Quality Gate (Batería de Tests)

El arnés valida cada paso mediante un ciclo de retroalimentación activa. Se amplía la batería de 4 a 8 tests para cubrir integridad funcional total.

ID	Test	Descripción y Criterios de Aceptación
01	Integridad de Contexto	Verifica insumos necesarios. NUEVO: Auditoría de secretos (API Keys) y presencia de .env.
02	Validación Semántica	Valida el brief contra el schema. NUEVO: Detección obligatoria de activos de mapas y ubicación.
03	Consistencia Visual	Respeto estricto a los tokens de diseño (colores, tipografía, espaciado).
04	Responsividad	Verifica breakpoints (375px, 768px, 1280px). Control de overflow y legibilidad.
05	Accesibilidad	Cumplimiento WCAG 2.1: Contraste min. 4.5:1, Alt-text y áreas táctiles (44px).
06	Navegación y Flujo	EXPANDIDO: Status 200 en enlaces internos/externos. Validación de feedback interactivo.
07	Performance	Optimización de activos (WebP). NUEVO: Verificación de renderizado de mapas dinámicos.
08	SEO y Metadatos	NUEVO: Validación de etiquetas Title, Meta-description y OpenGraph.

3. Arquitectura de Agentes

Separación de roles para eliminar el sesgo de auto-evaluación y garantizar la objetividad técnica.

Agente 1: The Builder (Productor)

- **Mandato:** Generar código funcional en `src/` y `dist/`.
- **Responsabilidad v3.1:** Lógica de navegación, integración de mapas y configuración de entornos.
- **Limitación:** No puede completar tareas sin aprobación externa.

Agente 2: The Tester (Auditor)

- **Mandato:** Ejecutar la batería completa de 8 tests.
- **Ciclo de corrección:** Dispara el refactor del Builder mediante reportes detallados.
- **Output v3.1:** Genera `test-report.json` (técnico) y `audit-summary.md` (ejecutivo).

4. Flujo de Trabajo (Lifecycle v3.1)

Fase	Agente	Acción	Resultado
1. Ingesta	Builder	Carga de Core + Contexto + .env	Límites definidos
2. Validación	Tester	context.md vs schema	Brief validado/rechazado
3. Producción	Builder	Generación de código y activos	Prototipo funcional
4. Auditoría	Tester	Ejecución de tests 01-08	Reportes de calidad
5. Refactor	Builder	Corrección técnica y feedback humano	Producto final pulido
6. Re-ingesta	Builder	Cambio de scope: actualización de contexto	Ciclo iterativo validado

5. Especificaciones de Control

- **Changelog Automático:** Registro histórico con timestamp y hash del contexto para trazabilidad absoluta.
- **Schema de Validación:** Filtro de entrada para asegurar que el Builder tenga toda la información necesaria (objetivos, audiencia, restricciones).
- **Audit Summary:** Documento puente que traduce la calidad técnica en valor de negocio para el cliente final.